



ASSIGNMENT NO 1- FALL 2025

Course Title:	Operating System			Course Code:	CSC323	Credit Hours:	3(2,1)
Course	Engr. Ahmad Mudassir			Program	BCE		
Semester:	5 th	Batch:	FA23-BCE	Section:	A,B	Date:	19 th October 2025
Deadline	24 th October 2025			Maximum Marks:	50		
Student's Name:				Reg. No.	CUI/	/LHR	

Important Instructions / Guidelines:

Note: Assignments must reflect each student's individual work. Any instance of plagiarism, duplication, or copying from other students or online sources will result in a zero (0) grade for all parties involved.

Assignment Title: Concurrent Directory Analyzer using Threads and Synchronization

Objective

To apply concepts of process creation, thread management, inter-process communication, and synchronization in a real-world file system scenario.

Assignment Description

Design and implement a Concurrent Directory Analyzer program in C using POSIX system calls and pthread library.

The program should:

1. Take a directory path as a command-line argument.
2. Create one thread per subdirectory (recursively) using pthreads.
3. Each thread should use opendir() and readdir() to access files, count number of files, total size, and subdirectories.
4. Use mutexes to safely update global statistics.
5. The main thread waits for all threads using pthread_join().
6. Print summary report with total statistics.

Bonus (Advanced Tasks)

- Add process-level concurrency using fork() and pipes for interprocess communication.
- Implement a log writer thread that periodically writes statistics to a log file using write() system call.

Key Concepts Tested

- Process creation (fork(), exec(), wait())
- Thread creation and synchronization (pthread_create, pthread_join, pthread_mutex_t)
- Directory traversal (opendir(), readdir(), closedir())
- File I/O (read(), write())
- Interprocess communication (optional pipes)
- Resource sharing and protection (mutex/semaphore)

Deliverables

1. Source code files (.c)
2. Report including objective, working principle, flowchart, and output screenshots.
3. (Optional) Demo video or screenshot of program execution.

Test Case Table

Test Case ID	Input Directory	Description / Condition	Expected Output	Evaluation Criteria
TC1 – Basic	/testdir1	Directory with 5 files and 1 subdirectory.	Total files = 5, Total directories = 1, Total size matches actual.	Counts match; output format correct.
TC2 – Nested Recursion	/testdir_nested	Nested directories with multiple levels.	All nested files counted correctly.	Recursive traversal works; no missing files.
TC3 – Concurrency Stress	/testdir_concurrent	5 large subdirectories processed by threads.	Threads execute concurrently; total time < sequential.	Parallel execution verified; no data corruption.
TC4 – Empty / Invalid	/emptydir or /nonexistent	Empty directory or invalid path.	Prints “No files found” or error message.	Graceful handling; no crash.
TC5 – Performance	/testdir_large	Directory with 1000+ files.	Correct output within 2s (8 threads).	Improved performance; correct synchronization.

Supporting Test Setup Scripts

Use the following Bash commands to create test directories to run test cases

```
mkdir -p testdir1/sub
```

```
touch testdir1/file{1..5}.txt
```

```
touch testdir1/sub/file6.txt
```

```
mkdir -p testdir_nested/sub1/sub2/sub3
```

```
touch testdir_nested/sub1/file1.txt testdir_nested/sub1/sub2/file2.txt
```

```
mkdir -p testdir_concurrent/{A,B,C,D,E}
```

```
for d in testdir_concurrent/*; do touch $d/file{1..20}.txt; done
```

```
mkdir testdir_large
```

```
for i in {1..1000}; do echo "Hello" > testdir_large/file$i.txt; done
```

```
mkdir emptydir
```

Evaluation Criteria (Rubric)

Component	Marks	Description
Process creation & synchronization	10	Correct usage of fork/exec/wait
Thread creation and synchronization	15	Correct usage of pthreads, mutexes
Directory traversal logic	10	Correct use of opendir/readdir
IPC or file logging (bonus)	5	Correct pipe or I/O usage
Report & Output correctness	10	Clarity, correctness, readability
Total	50	